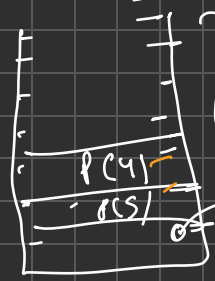
[Time & space complexity]

n to 1

→ Finish

```
void print(int n)
{
    if (n==0)
        return;

    cout<< n;
    print(n-1);
}
```

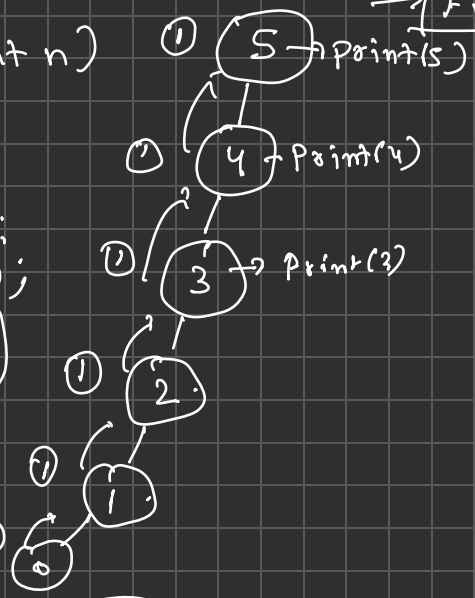
$O(n)$ $\Theta(n)$



max stack size

call int main()

print(5)



```
for(int i=5; i>0; i--)
    cout<<i;
```

$\boxed{O(n)}$

$\boxed{O(1)}$

$\boxed{\text{Loop}}$

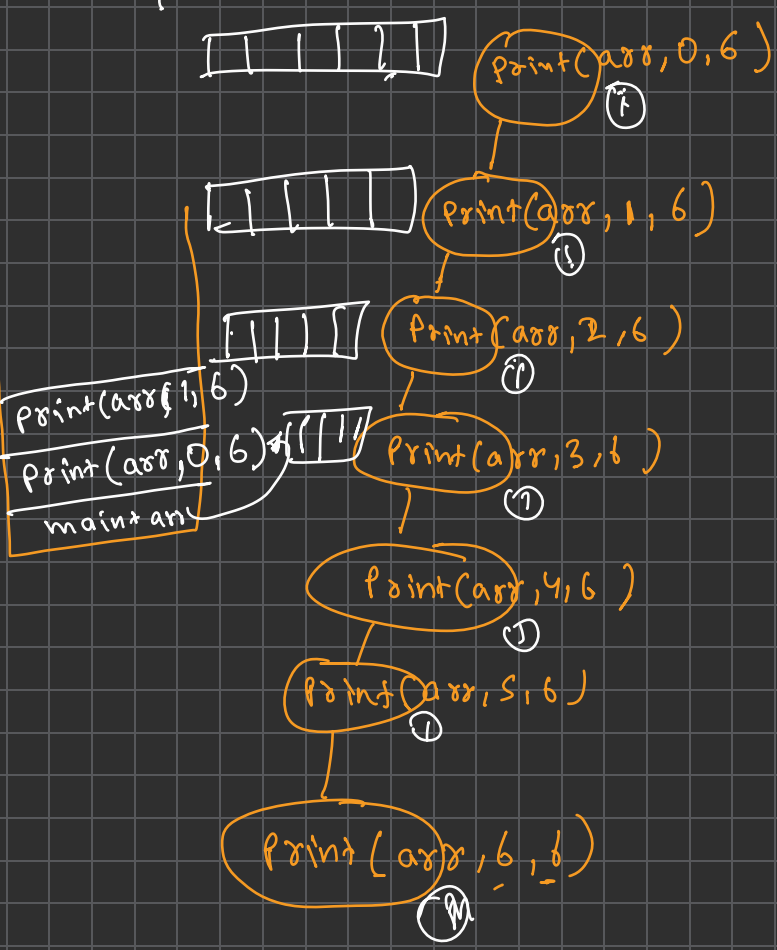
vector<int> arr:

→ | 6 | 10 | 5 | 18 | 23 | 9 |

← Print Recursiv

6 10 5 18 23 9

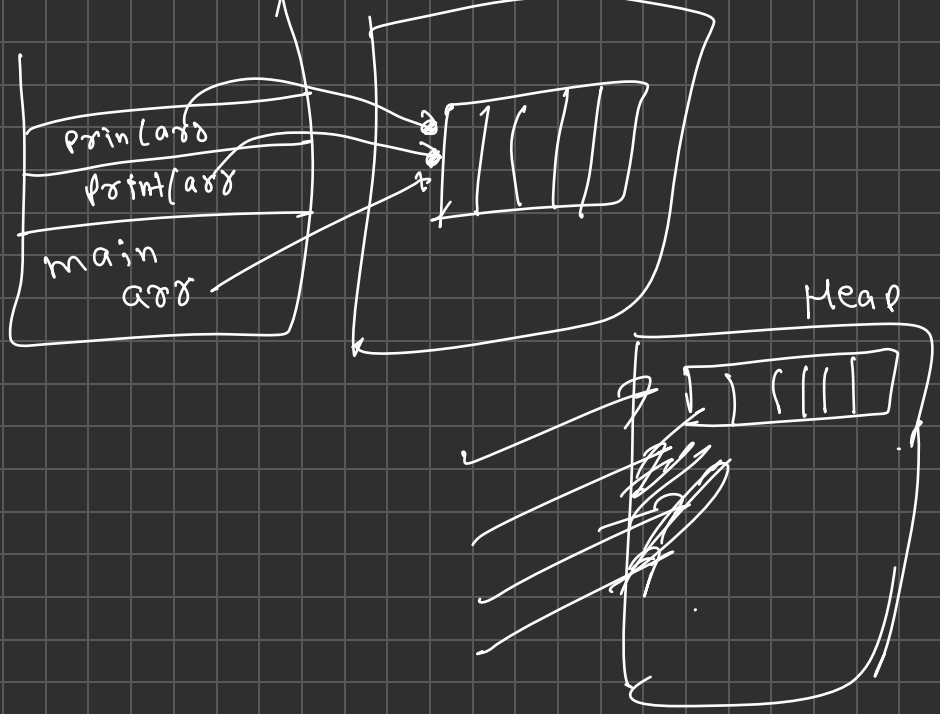
```
void print(vector<int> &arr, int index,
           int n)
{
    if (index==n)
        return

    cout<< arr[index];
    print(arr, index+1, n);
}
```



T. Comp = O(n)

$O(n^2)$

$O(n)$


print(arr)
print(arr)
main
arr
Heap

$F(n) = f(n-1) + f(n-2)$ ⇒ Recursiv

```
int Fibonacci( int n )
{
    if (n <= 1)
        return n;

return fibonacci(n-1) + fibonaci(n-2)
```

0 1 1 2 3 5

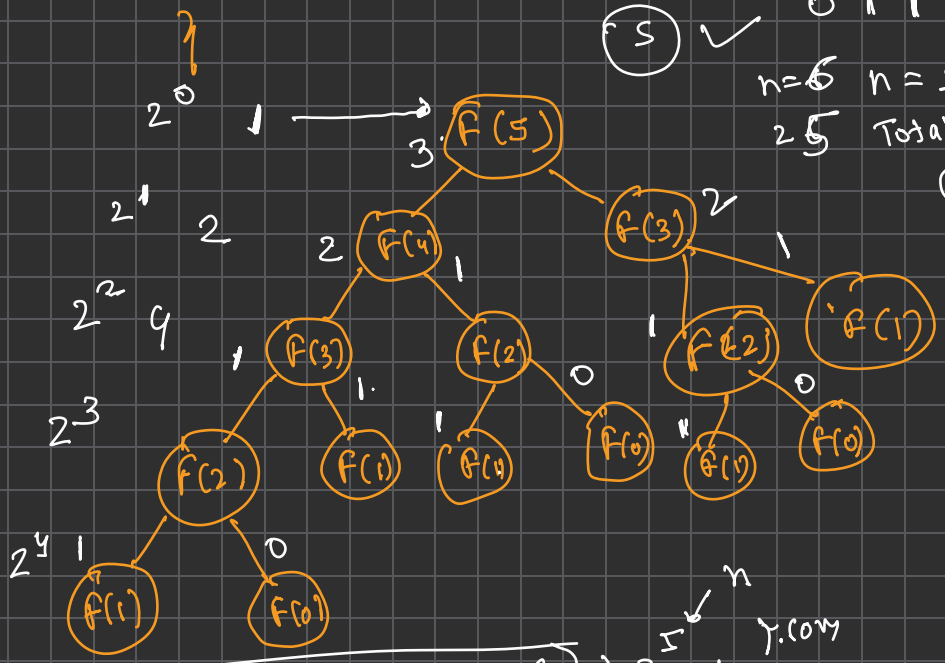
5

n=6 n=5

25 Total ⇒ 15

3n

O(n)



$$\boxed{2^0 + 2^1 + 2^2 + 2^3 + 2^4} \Rightarrow 2^5 - 1$$

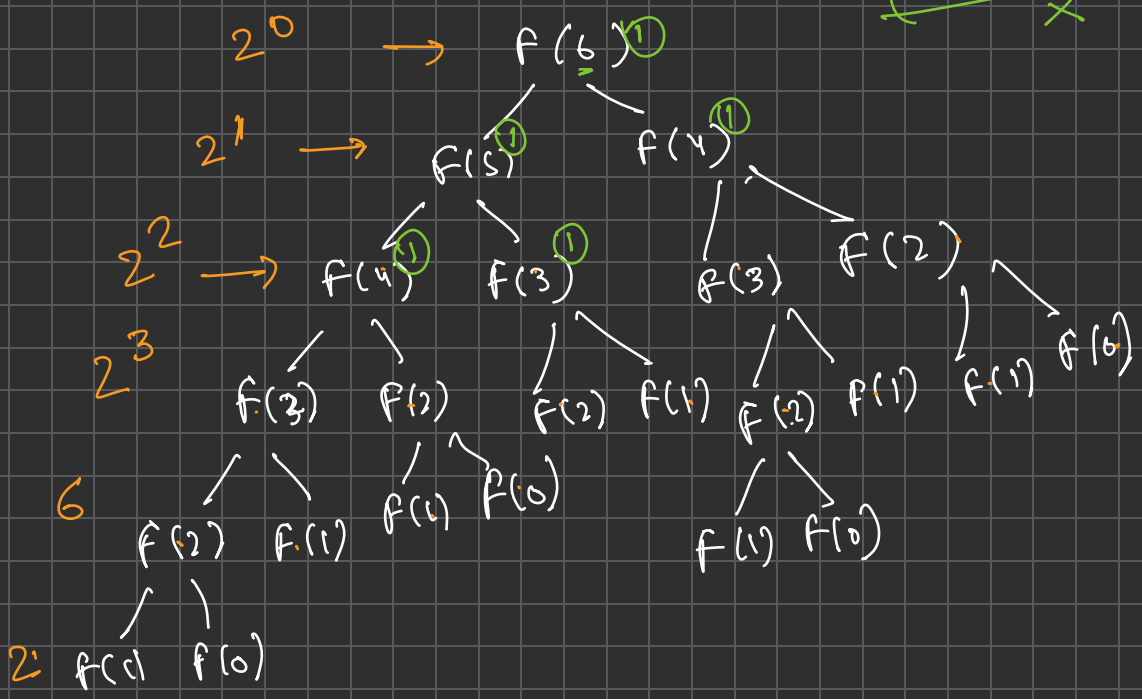
$$\Rightarrow 2^n - 1$$

$F(6)$ $O(2^n)$

15

F(5) F(4)

$O(2^n)$



$$\left(\underbrace{2^0 + 2^1 + 2^2 + 2^3}_{} + 6 + 2\right)$$

$$2^4 - 1 + 6 + 1$$

$$2^{6-2} + 6$$

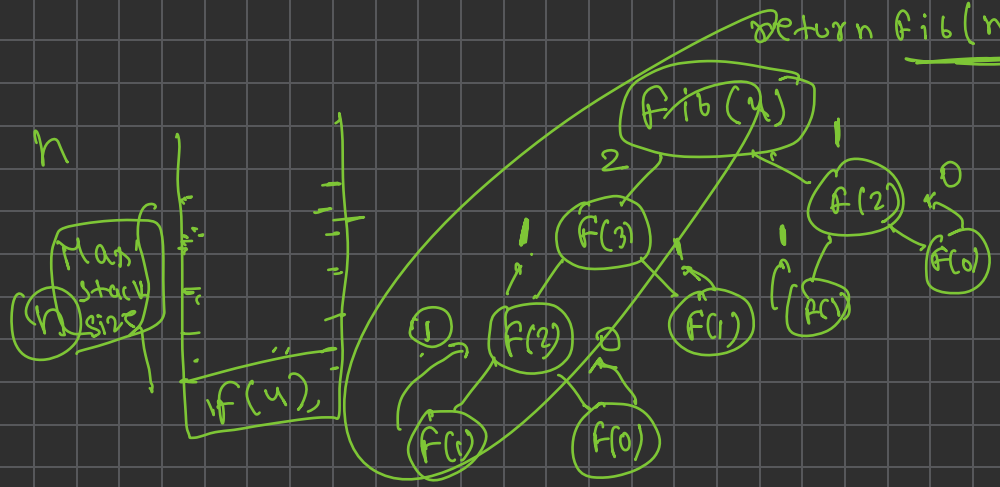
$$2^{n-2} + \cancel{6}$$

$$2^n$$

Time complexity

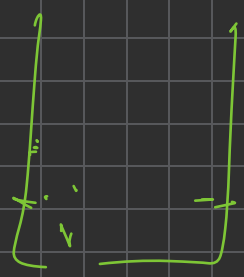
memory Requ

```
int fib(int n)
{   if (n<=1)
        return n;
    return fib(n-1)+fib(n-2)
```



Time Complex: [Total node, sum]

Space Complex: Longest path:



```
void print(int n)
{   :
    cout<<n; x
}
}
```

```
main()
{
    print(5)
```

n^2

```
int main()
{
    int n;
    cin>>n;
    int *p = new int[n];
    int *p2 = new int[n];
```

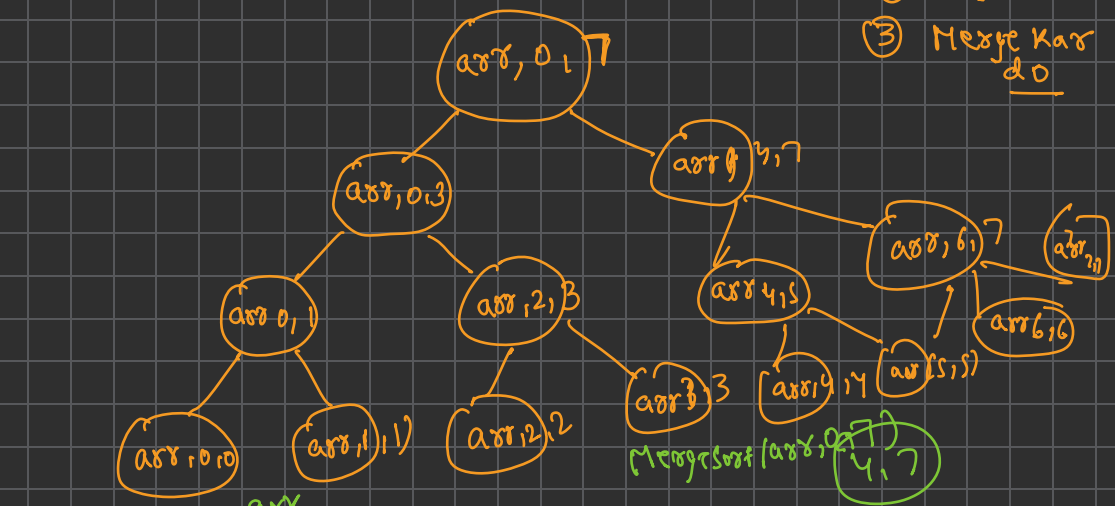
delete [] p

delete [] p2

n

0	1	2	3	4	5	6	7
10	3	7	2	19	8	3	20

1. Left part
2. Right part
3. Merge Kar do



0	1	2	3	4	5	6	7
2	3	7	10	3	8	19	20

arr

0, 3

MergeSort(arr, 0, 3)

MergeSort(4, 7)

update

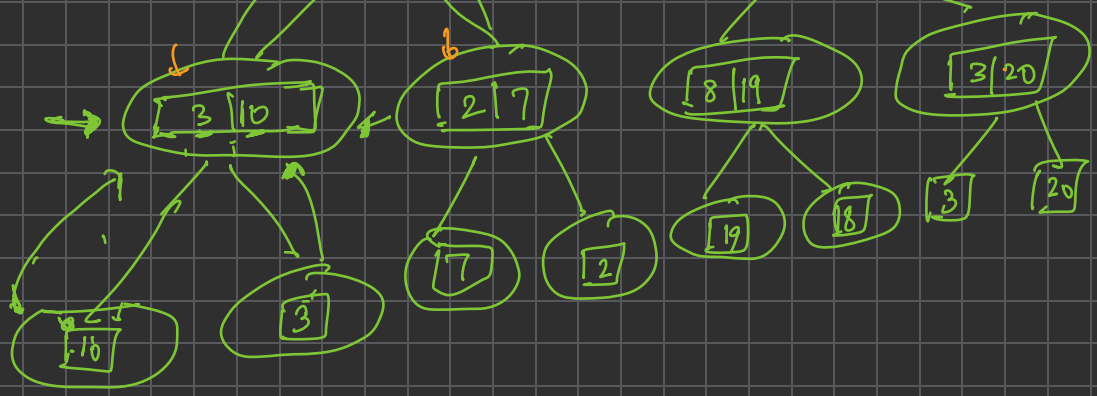
arr

start

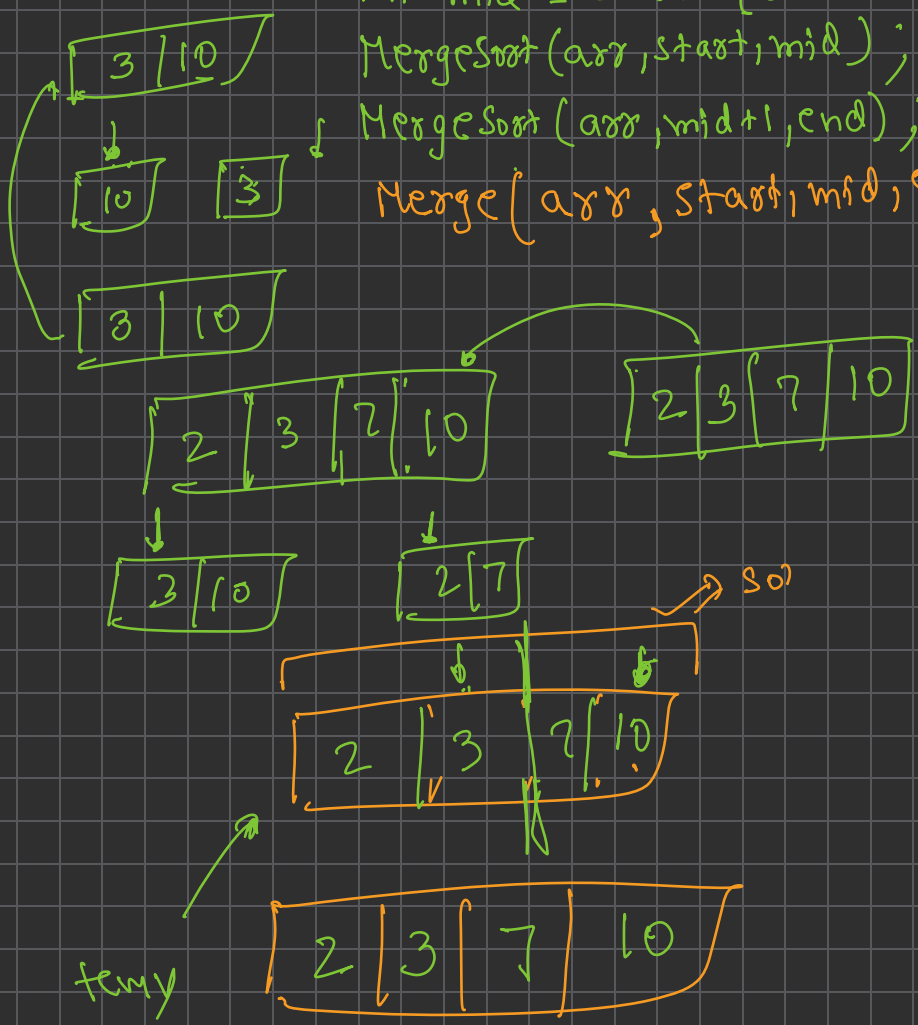
end

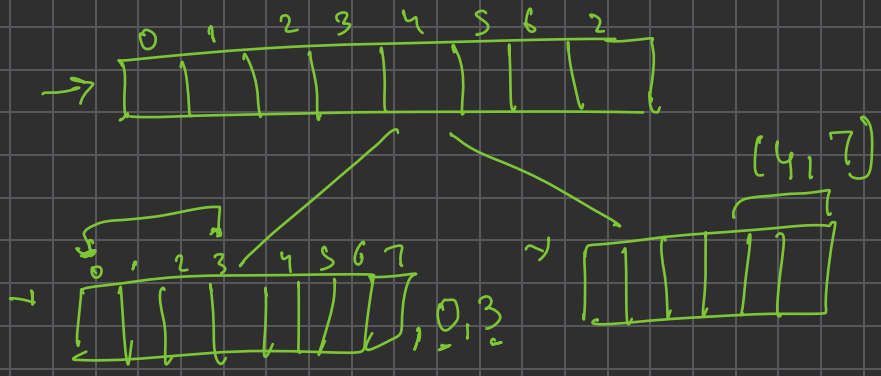
start

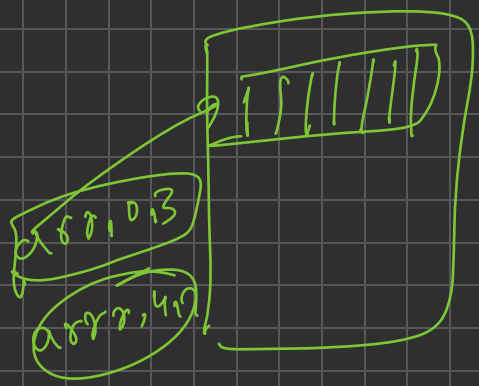
end

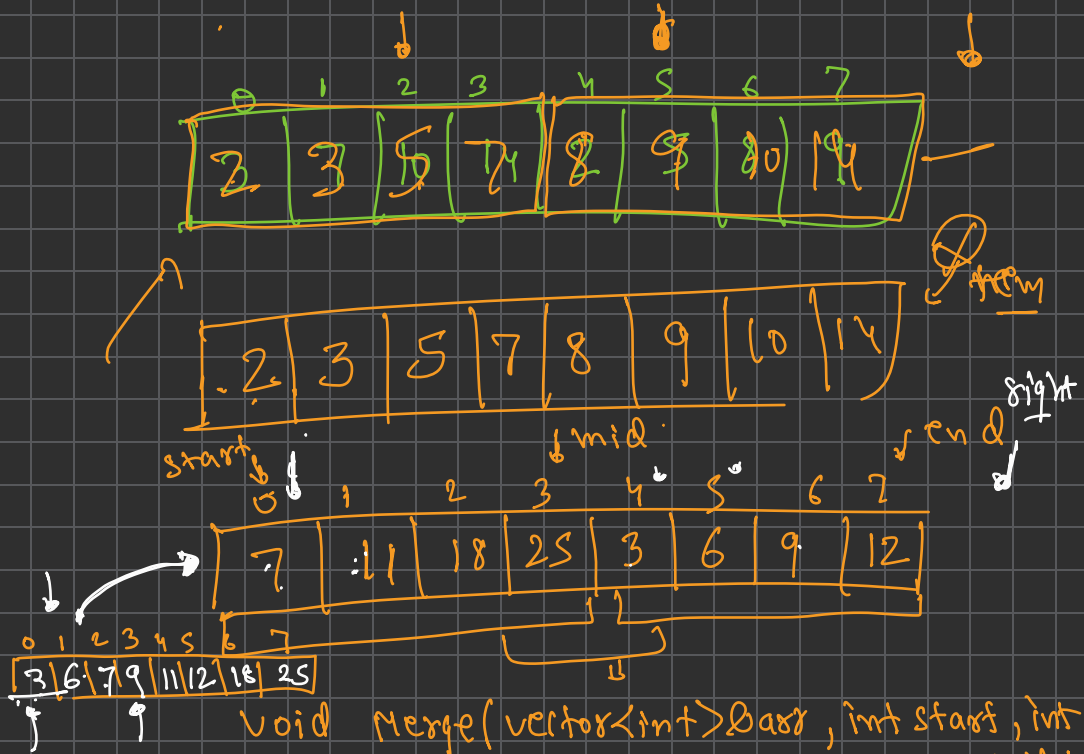


```
void MergeSort( vector<int> &arr, int start, int end)
{
    if (start >= end)
        return ;

    int mid = start + (end-start)/2
    MergeSort(arr, start, mid) ;
    MergeSort(arr, mid+1, end) ;
    Merge(arr, start, mid, end);
```




0 1 2 3 4 5 6 7
0 1 2 3 4 5 6 7
0,3
(4,7)


0,3
4,7




```
void Merge(vector<int>&arr, int start, int mid, int end)
{
    vector<int> temp(end-start+1)
    int left => start, right = mid+1;
      int index=0
    while(left<=mid && right<=end)
    {
        if (arr[left] <= arr[right])
        {
            temp[index] = arr[left];
              index++; left ++;
        }
```

```
else
{
    temp[index] = arr[right];
    index++, right++;
}
}

// Left wale part mein clen
while( left <= mid )
{ temp[index++] = arr[left++];
}
// Right wale
while (right <= end )
{ temp[index++] = arr[right++];
}
    index = 0;
for ( i = start ; i <= end ; i++)
{   arr[i] = temp [index]
        index++;
}
```